FLUTTER COURSE · SESSION 4 OF 15

Navigation & Forms

Until now every app you built had a single screen. Today you learn to move between screens, send data to a screen and get an answer back, and collect input from the user with forms that check what they type.

BY THE END OF TODAY YOU CAN

Build a multi-screen app that passes data forward, receives results back, and uses a validated form to add new items.

How to run today's apps

In VS Code, open **Run and Debug** (`Cmd+Shift+D`), pick an app from the dropdown under **Session 4**, and press **F5**. It runs in debug mode, so the Widget Inspector and breakpoints work. From the terminal, use:

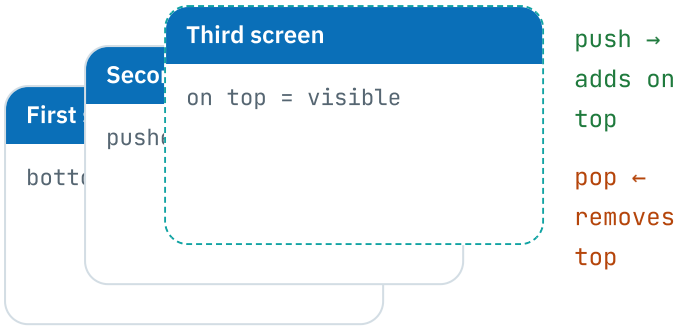
```
flutter run -t lib/day4_demos/demo1_push_pop.dart
flutter run -t lib/contact_book/main.dart
```

Several demos print to the **Debug Console**. Keep it open today; it shows exactly when Flutter rebuilds a widget or disposes it.

The concepts, in the order we cover them

`demo1` Navigator push and pop

The Navigator keeps screens in a **stack**, like a pile of plates. `push` puts a new screen on top. `pop` removes the top screen and shows the one underneath. The back arrow in the AppBar is just a `pop` .




```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => const SecondScreen()),
);

Navigator.pop(context);   // go back one screen
```

---

demo2 Passing data to a new screen

A screen is just a widget, so you send it data through its **constructor**, the same way you passed a dish to `DishCard` last session.




```
class ProductDetailScreen extends StatelessWidget {
  const ProductDetailScreen({super.key, required this.product});
  final Product product;   // the data this screen needs
  ...
}

// on tap in the list:
Navigator.push(context, MaterialPageRoute(
  builder: (context) => ProductDetailScreen(product: product),
));
```

---

demo3 Returning a result from a screen

`Navigator.push` returns a `Future`. If you `await` it, your code waits until the next screen closes. That screen sends a value back with `Navigator.pop(context, value)`.




```dart
final chosen = await Navigator.push<String>(
  context,
  MaterialPageRoute(builder: (context) => const SizeScreen()),
);
if (!mounted) return;        // the screen may be gone after an await
if (chosen == null) return;   // user pressed back
setState(() => _size = chosen);
```

- If the user presses the back arrow, the result is `null`. Always handle that case.
- After an `await`, check `mounted` before you use `context` or call `setState`.

---

demo4 Named routes

Instead of building a `MaterialPageRoute` every time, give each screen a name in one place and navigate by name.

```
class AppRoutes {
  static const home = '/';
  static const profile = '/profile';
}

MaterialApp(
  initialRoute: AppRoutes.home,
  routes: {
    AppRoutes.home: (context) => const HomeScreen(),
    AppRoutes.profile: (context) => const ProfileScreen(),
  },
)

Navigator.pushNamed(context, AppRoutes.profile, arguments: 'Priya');

// inside ProfileScreen:
final name = ModalRoute.of(context)!.settings.arguments as String;
```

- Keep route names as constants. A typo then becomes a compile error instead of a crash.
- `arguments` has no type, so you have to cast it. For passing data, constructors are safer.

`demo5` setState() and how rebuilds work

Changing a variable does not change the screen. `setState` changes the value **and** tells Flutter to run `build()` again so the screen shows the new value.

```
// ❌ value changes, screen does not
_count++;

// ✅ value changes and build() runs again
setState(() => _count++);
```

In the demo, tap “+1 without setState” three times. Nothing changes on screen. Then tap “+1 with setState” and the number jumps by four. The console also shows that `const` widgets are not rebuilt.

demo6 StatefulWidget lifecycle: initState and dispose



A StatefulWidget is split into two classes. The widget itself (`SecondsCounter`) only describes what to show and is recreated often. Its **State** object (`_SecondsCounterState`) holds the values that change, like `_seconds` , and stays alive while the widget is on screen. `createState()` is where Flutter makes that State object. You never call it yourself.

```
@override
State<SecondsCounter> createState() => _SecondsCounterState();
```

In the demo, “Remove counter” destroys the State object. “Add counter” calls `createState()` again and makes a new one, which is why the count starts again from 0.

```
@override
void initState() {
  super.initState();          // super FIRST
  _timer = Timer.periodic(...);  // start things here
}

@override
void dispose() {
  _timer.cancel();              // stop things here
  super.dispose();             // super LAST
}
```

Anything you start in `initState` (timers, controllers, listeners) must be stopped in `dispose` . Otherwise it keeps running after the widget is gone, and you get `setState() called after dispose()` .

---

demo7 TextField and TextEditingController

A `TextEditingController` lets your code read, change and clear the text in a `TextField` .

```
final _nameController = TextEditingController();

TextField(
  controller: _nameController,
  decoration: const InputDecoration(labelText: 'Your name'),
  keyboardType: TextInputType.name,
)

_nameController.text      // read what the user typed
_nameController.clear()    // empty the field

@override
void dispose() {
  _nameController.dispose();   // always dispose controllers
  super.dispose();
}
```

- `keyboardType` picks the keyboard: `phone`, `emailAddress`, `number`.
- `obscureText: true` hides a password.
- `onChanged` runs on every keystroke. `onSubmitted` runs when the user presses Done.

`demo8` Form, TextFormField and validators

A `Form` groups fields together. A `GlobalKey<FormState>` gives you a handle on the form, so one call can check every field.



```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    validator: (value) {
      if (value == null || value.isEmpty) return 'Enter a phone number';
      if (value.length != 10) return 'Phone number must be 10 digits';
      return null;   // null means valid
    },
  ),
)

// on Save:
if (_formKey.currentState!.validate()) {
  // every field is valid
}
```

- A validator returns `null` when the value is fine, or an error message when it isn't.
- `validate()` runs every validator, shows the messages, and returns `true` only if all fields pass.
- `autovalidateMode: AutovalidateMode.onUserInteraction` re-checks a field while the user types.

The project: Contact Book App

A contact list with three screens. The list shows every contact with an initials avatar. Tapping a contact opens its details. The “Add contact” button opens a form that checks the name, a 10-digit phone number and the email, then sends the new contact back to the list. The detail screen can also delete a contact by sending `true` back.



```
lib/contact_book/
├─ main.dart · starts the app, sets the theme
├─ validators.dart · name, phone and email checks
├─ models/contact.dart · the Contact class
├─ widgets/contact_avatar.dart · initials circle, used on 2 screens
└─ screens/
   ├─ contact_list_screen.dart
   ├─ contact_detail_screen.dart
   └─ add_contact_screen.dart
```

Where each concept appears

FILE	WHAT TO LOOK FOR
`contact_list_screen.dart`	`await Navigator.push<Contact>`, the `mounted` check, `setState` to add or remove, a SnackBar, and an empty state when there are no contacts
`contact_detail_screen.dart`	Receives a `Contact` through its constructor. Delete calls `Navigator.pop(context, true)`.
`add_contact_screen.dart`	`Form`, `GlobalKey<FormState>`, three controllers created and disposed, `validate()`, and `Navigator.pop(context, contact)`
`validators.dart`	Validators as plain functions, so the form can use them and tests can check them
`contact_avatar.dart`	One small widget reused on two screens

The project also has tests in `test/contact_book_test.dart`. Run `flutter test` to see them check the validators and the full add → open → delete flow.

Errors you will see today

> **`The number doesn't change on screen`**
>
> You changed a variable without `setState`.

> **`"setState() called after dispose()"`**
>
> A timer or listener is still running after the widget was removed. Cancel it in `dispose`, and check `mounted` after every `await`.

> **`"Null check operator used on a null value" at currentState!`**
>
> You forgot `key: _formKey` on the `Form`.

> **`The result is always null`**
>
> The screen called `Navigator.pop(context)` without a value, or the user pressed back.

> **`"type 'Null' is not a subtype of type 'String'"`**
>
> You opened a named route without `arguments`, but the screen casts them to a String.

> **`"Navigator operation requested with a context that does not include a Navigator"`**
>
> You used the `context` of the widget that builds `MaterialApp`. Navigate from a screen inside `home:` instead.

Practice task (in class)

Build a 3-screen app about books, students or anything you like.

- [ ] A list screen built with `ListView.builder`.
- [ ] Tapping an item opens a detail screen that receives the item through its constructor.
- [ ] A "+" button opens a form screen. Every field has a validator.
- [ ] Saving returns the new item with `Navigator.pop(context, item)`, and the list adds it with `setState`.
- [ ] All controllers are disposed.

Homework: extend the Contact Book

- [ ] Add an **Edit** button on the detail screen. It opens the form already filled in and returns the updated contact.

- [ ] Add an optional **Company** field that is allowed to be empty.
- [ ] Add a search `TextField` above the list that filters contacts as you type.
- [ ] Switch the app to named routes with an `AppRoutes` class.
- [ ] Commit your work and push it to your GitHub repo.

Check yourself

<details>
<summary>How does a screen send data back to the screen that opened it?</summary>

It calls `Navigator.pop(context, value)`. The screen that opened it receives the value from `await Navigator.push(...)`.
</details>

<details>
<summary>What does `await Navigator.push(...)` give you if the user presses back?</summary>

`null`. That's why you check for null before using the result.
</details>

<details>
<summary>What happens if you change a variable without calling `setState`?</summary>

The variable changes, but `build()` doesn't run, so the screen keeps showing the old value.
</details>

<details>
<summary>Why must you dispose a `TextEditingController`?</summary>

It holds resources and listeners. If you don't dispose it, it stays in memory after the screen is gone.
</details>

<details>
<summary>What should a validator return when the value is valid?</summary>

`null`. Any String it returns is shown as the error message.
</details>

NEXT SESSION

Session 5 · State Management with Riverpod. Why `setState` gets hard to manage as an app grows, and how Riverpod shares state between screens. You'll build a Pizza Order app.